```java

import java.util.*;

public class ZeroOneKnapsack {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Number of items: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Invalid input: At least one item is required.");
            sc.close();
            return;
        }

        int[] weight = new int[n];
        int[] value = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Weight and value of item " + (i + 1) + ": ");
            weight[i] = sc.nextInt();
            value[i] = sc.nextInt();

            if (weight[i] <= 0 || value[i] < 0) {
                System.out.println("Invalid item data: Weight must be positive and value must be non-negative.");
                sc.close();
                return;
            }
        }

        System.out.print("Knapsack capacity: ");
        int capacity = sc.nextInt();

        if (capacity < 0) {
            System.out.println("Invalid capacity: Knapsack capacity cannot be negative.");
            sc.close();
            return;
        }

        if (capacity == 0) {
            System.out.println("Maximum profit: 0");
            System.out.println("No items selected because the knapsack capacity is zero.");
```

```
            sc.close();
            return;
        }


        int[][] dp = new int[n + 1][capacity + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 0; w <= capacity; w++) {
                if (weight[i - 1] <= w) {
                    dp[i][w] = Math.max(
                        dp[i - 1][w],
                        value[i - 1] + dp[i - 1][w - weight[i - 1]]
                    );
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }

        System.out.println("Maximum profit using Dynamic Programming: "
                + dp[n][capacity]);

        System.out.print("Selected items: ");
        int w = capacity;
        boolean found = false;

        for (int i = n; i > 0; i--) {
            if (dp[i][w] != dp[i - 1][w]) {
                System.out.print("Item " + i + " ");
                w -= weight[i - 1];
                found = true;
            }
        }

        if (!found) {
            System.out.print("None");
        }

        System.out.println();

        int totalWeight = 0;

        for (int i = 0; i < n; i++) {
            if (dp[i + 1][capacity] != dp[i][capacity]) {
                totalWeight += weight[i];
            }
        }
```

```
98
91
92          System.out.println("Knapsack capacity: " + capacity);
93          System.out.println("Maximum profit calculated successfully.");
94
95          sc.close();
96      }
97  }
```

Output :

